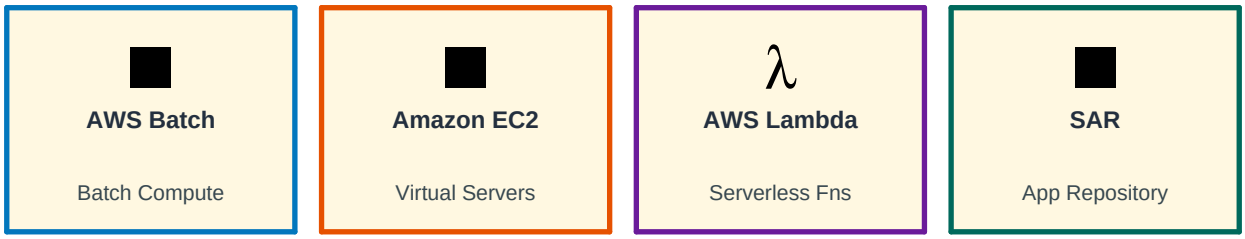
AWS Compute Services

Comprehensive Learning Guide

AWS Batch · Amazon EC2 · AWS Lambda · Serverless Application Repository

Beginner-Friendly • In-Depth • Practical • Hands-On



AWS Solutions Architect Training Series • Version 1.0 • 2025

Table of Contents

■ AWS Batch

Run Hundreds of Thousands of Batch Computing Jobs Effortlessly

1. Simple Introduction

AWS Batch is a **fully managed batch computing service** that enables developers, scientists, and engineers to easily and efficiently run hundreds of thousands of batch computing jobs on AWS. It dynamically provisions the optimal quantity and type of compute resources (such as CPU or memory-optimised instances) based on the volume and specific resource requirements of the batch jobs submitted.

Why Does It Exist? Batch workloads — jobs that run start-to-finish without human interaction — are everywhere: nightly financial reconciliation, genomics sequencing, media rendering, scientific simulations. Setting up the infrastructure to run these efficiently (queues, auto-scaling, spot instances, scheduling) is complex and time-consuming. AWS Batch removes that burden entirely.

■ **Real-World Analogy:** Think of a **large commercial printing press**. Publishers (you) send print jobs (batch jobs) of varying sizes to the print facility (AWS Batch). The facility manager (AWS Batch scheduler) organises the jobs into queues, assigns them to the right press machines (compute resources), and scales up machines for rush orders. You never worry about how many presses are running — you just submit jobs and collect results.

Business Problem It Solves:

- Eliminates need to manage servers, clusters, or job schedulers manually
- Automatically scales compute capacity up and down based on job demand
- Optimises costs using Spot Instances for fault-tolerant workloads
- Handles job dependencies — Job B waits for Job A to complete first
- Supports GPU-intensive workloads (ML training, video rendering) natively

2. Core Concepts

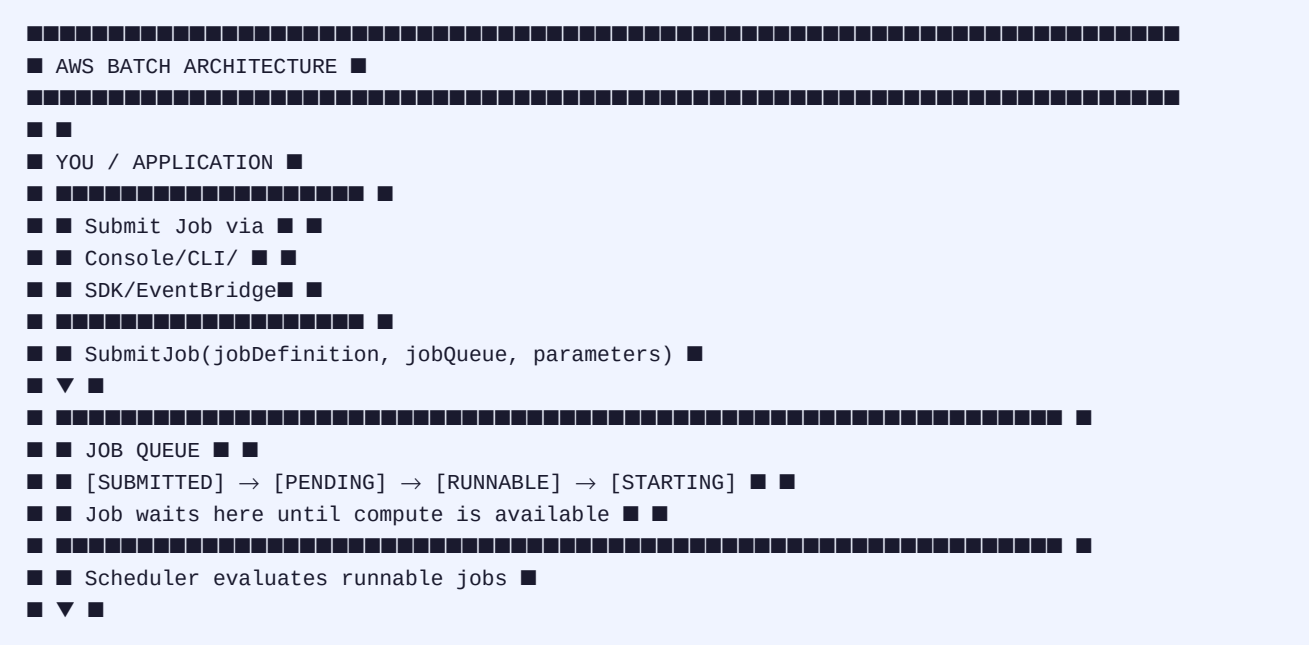
Term	Definition	Example
Job	The fundamental unit of work — a containerised task	Process 10,000 genomics files
Job Definition	Blueprint specifying container image, CPU, memory, env vars	cpu:4, memory:8GB, image:"myecr/genomics:v2"
Job Queue	Where submitted jobs wait before being scheduled	high-priority-queue, low-priority-queue
Compute Environment	The set of managed or unmanaged EC2/Fargate resources	Spot Fleet of c5 instances, or Fargate tasks
Scheduler	AWS Batch component that matches jobs to compute	Picks cheapest available Spot instance for the job
Array Job	A single job that spawns N child jobs in parallel	Process 1000 images — 1000 child jobs, one per image

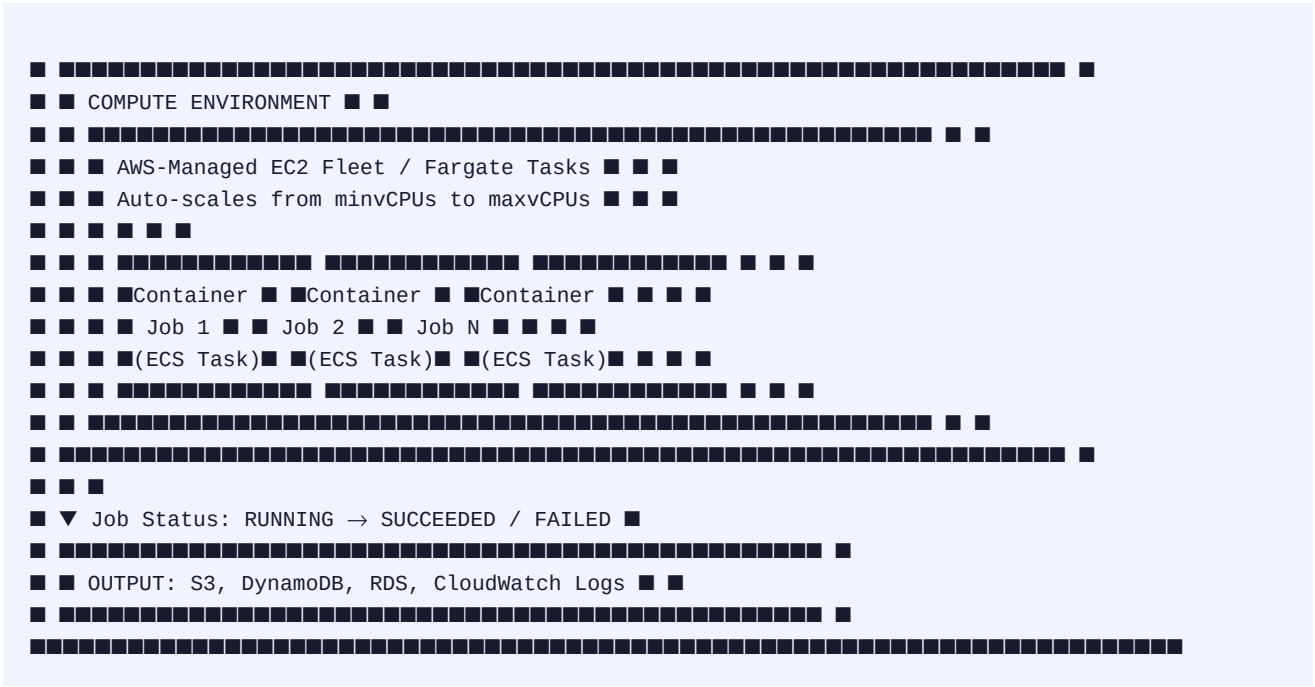
Term	Definition	Example
Job Dependencies	Jobs that must succeed before another starts	Transcode job waits for Download job to finish
Managed CEnv	AWS manages instance lifecycle, scaling, patching	Most common — let AWS handle it
Unmanaged CEnv	You manage the EC2 instances; AWS only schedules	Specialised hardware or licensing requirements
Fair Share Policy	Allocate compute proportionally across job queues	Team A gets 60%, Team B gets 40% of capacity
Priority	Integer priority for a job queue (higher = sooner scheduled)	Production queue priority 100 vs Dev queue priority 10
Multi-Node Parallel	Single job runs across multiple instances (MPI workloads)	Distributed ML training across 16 EC2 nodes

Compute Environment Types

Type	EC2 On-Demand	EC2 Spot	Fargate	Fargate Spot
Cost	Highest	60-90% cheaper	Pay per vCPU/mem	~70% off Fargate
Interruption	None	Can be reclaimed	None	Can be reclaimed
Best for	Critical deadlines	Fault-tolerant batch	Short jobs <4hr	Cost-optimised short jobs
GPU support	■ Yes	■ Yes	■ No	■ No
Setup time	Minutes	Minutes	Seconds	Seconds

3. Architecture & Working





Job Lifecycle (Step-by-Step)

Step	State	What Happens
1	SUBMITTED	Job accepted; checked against job definition and dependencies
2	PENDING	Waiting for dependent jobs to finish (if any dependencies set)
3	RUNNABLE	Ready to run; scheduler seeking compute capacity
4	STARTING	Compute resources allocated; container image being pulled
5	RUNNING	Container executing — your code is running
6	SUCCEEDED	Exit code 0 — job completed successfully
6	FAILED	Non-zero exit code — Batch can auto-retry N times

4. Features

Feature	Description	Benefit
Automatic Scaling	Scales compute from 0 to thousands of vCPUs based on queue depth	No idle resources; no under-provisioning
Spot Instance Integration	Use EC2 Spot for up to 90% cost savings; auto-retries on interruption	Massive cost reduction for fault-tolerant jobs
Array Jobs	Run thousands of identical parallel child jobs from one submission	Process large datasets in parallel trivially
Job Dependencies	DAG-based dependencies — jobs wait for predecessors to succeed	Complex multi-stage pipelines with guaranteed order
GPU Support	Native support for GPU instance types (p3, g4dn, p4d)	ML training, video rendering, scientific simulations

Feature	Description	Benefit
Multi-Node Parallel	MPI-style jobs spanning multiple EC2 instances with shared networking	HPC (High Performance Computing) workloads
Fair Share Scheduling	Proportional resource allocation across teams/queues over time	Fair multi-team usage without starvation
Secrets Manager Integration	Inject secrets into jobs without hardcoding credentials	Secure, auditable credential management
EFS / FSx Integration	Mount shared file systems into batch job containers	Share large datasets across thousands of parallel jobs
Retry Strategy	Automatically retry failed jobs up to 10 times with exit code filters	Handle transient failures without manual intervention
CloudWatch Integration	Job metrics, logs, and alarms in CloudWatch	Full observability into batch workloads

5. Real-World Use Cases

Industry	Use Case	Details
Genomics / Life Sciences	DNA Sequencing Pipeline	Process millions of genomics reads in parallel; each sample = one array job child
Financial Services	End-of-Day Risk Calculations	Run Monte Carlo simulations across thousands of portfolios nightly
Media & Entertainment	Video Transcoding	Convert uploaded videos to 10 formats simultaneously using GPU instances
E-Commerce	Product Image Processing	Resize, watermark, and compress millions of product images on upload
Healthcare	Medical Image Analysis	Run ML inference on CT/MRI scans — one job per scan, thousands in parallel
AI / ML	Hyperparameter Tuning	Launch 500 training jobs with different parameters simultaneously
Climate / Science	Weather Simulation	Run atmospheric models across multi-node clusters using MPI
Retail	ETL Data Processing	Nightly transformation of terabytes of clickstream data
Gaming	Game Replay Processing	Analyse millions of player game replays to detect cheating or bugs

6. Practical Example

Scenario: Process 1,000 uploaded images in parallel — resize each to three sizes (thumb, medium, large) and save to S3.

Step 1 – Create a Job Definition:

```
aws batch register-job-definition \ --job-definition-name image-processor \ --type container \
--container-properties '{ "image": "123456789.dkr.ecr.us-east-1.amazonaws.com/img-proc:v1",
"vcpus": 2, "memory": 4096, "environment": [{"name":"OUTPUT_BUCKET","value":"my-resized-images"}],
"jobRoleArn": "arn:aws:iam::123456789:role/BatchJobRole" }'
```

Step 2 – Create a Compute Environment (Spot):

```
aws batch create-compute-environment \ --compute-environment-name spot-img-env \ --type MANAGED \
--compute-resources '{ "type": "SPOT", "bidPercentage": 60, "minvCpus": 0, "maxvCpus": 2000,
"instanceTypes": ["c5","m5"], "subnets": ["subnet-abc123"], "securityGroupIds": ["sg-xyz789"],
"instanceRole": "arn:aws:iam::123456789:instance-profile/ecsInstanceRole" }'
```

Step 3 – Submit an Array Job (1,000 child jobs):

```
aws batch submit-job \ --job-name process-1000-images \ --job-queue image-processing-queue \
--job-definition image-processor \ --array-properties '{ "size": 1000 }' \ --parameters
'{"inputBucket":"raw-images"}'
```

■ **Result:** AWS Batch launches up to 1,000 containers in parallel on Spot Instances. Each child job reads AWS_BATCH_JOB_ARRAY_INDEX (0–999) to know which image to process. Total cost is ~60–90% cheaper than On-Demand EC2.

7. Integrations

AWS Service	How It Integrates
Amazon S3	Jobs read input data from S3 and write results back — most common pattern
Amazon ECR	Container images for batch jobs stored and pulled from ECR
AWS Step Functions	Orchestrate Batch jobs in multi-step workflows with dependencies and error handling
Amazon EventBridge	Trigger Batch jobs on schedule or on events (e.g., S3 upload)
AWS Lambda	Lambda triggers Batch jobs; Batch job completion triggers Lambda
Amazon CloudWatch	Job metrics, logs, and custom alarms for monitoring
AWS Secrets Manager	Inject DB passwords, API keys into job containers securely
Amazon EFS / FSx	Mount shared file systems so parallel jobs access the same data
Amazon DynamoDB	Jobs write results/status to DynamoDB for downstream consumers
AWS IAM	Job roles grant containers least-privilege access to AWS services
Amazon SNS	Notify teams when jobs fail or succeed via SNS alerts
AWS Glue	Batch pre-processes raw data; Glue runs further ETL transformations

8. Comparison

Feature	AWS Batch	AWS Lambda	Amazon ECS	AWS Glue
Type	Managed batch scheduler	Serverless functions	Container orchestrator	Managed ETL
Max Duration	Unlimited	15 minutes	Unlimited	Unlimited

Feature	AWS Batch	AWS Lambda	Amazon ECS	AWS Glue
GPU Support	■ Yes	■ No	■ Yes	■ No
Scheduling	■ Built-in queues	Via EventBridge	Manual / ECS Scheduler	■ Triggers
Dependencies	■ Job DAGs	■ No	■ No	■ Yes
Spot Support	■ Native	N/A	■ Yes	■ Yes
Container	■ Required	■ (optional)	■ Required	■ No
Best For	Long batch workloads	Short event-driven	Always-on services	Data ETL only

■ **Rule of Thumb:** Use **AWS Batch** for compute-heavy, long-running batch jobs with complex dependencies. Use **Lambda** for quick event-driven tasks under 15 min. Use **ECS** for persistent containerised microservices. Use **Glue** for Spark-based ETL on AWS data stores.

■ Amazon EC2

Elastic Compute Cloud — Resizable Virtual Servers in the Cloud

1. Simple Introduction

Amazon EC2 (Elastic Compute Cloud) is AWS's **flagship virtual server service**. It lets you rent virtual computers in the cloud — called **instances** — in minutes, with the ability to scale from one instance to thousands and back. EC2 is the backbone of AWS and powers the vast majority of cloud workloads worldwide.

Why Does It Exist? Before cloud computing, companies had to buy physical servers, wait weeks for delivery, and then maintain them. EC2 changed everything: provision a powerful server in 60 seconds, pay only for what you use, and return it when done. It democratised computing power.

■ **Real-World Analogy:** EC2 is like a **hotel**. You don't buy the building — you rent a room for as long as you need it. You can rent a single standard room (small instance) or an entire floor of suites (large instances). You choose the room type (instance family), add-ons like parking (EBS storage), and check out (terminate) when done. The hotel manages the building; you manage your room.

Business Problems Solved:

- No upfront capital expenditure — rent instead of buy
- Scale compute in minutes — handle traffic spikes without planning ahead
- Choose from 750+ instance types optimised for CPU, memory, storage, GPU, networking
- Run any OS, any software — full root/admin access to the instance
- Global reach — launch instances in 30+ AWS regions worldwide

2. Core Concepts

Term	Definition	Example
Instance	A virtual server running in AWS	t3.medium running Ubuntu 22.04
AMI	Amazon Machine Image — snapshot of an OS + software config	Official Amazon Linux 2023 AMI, or your custom golden AMI
Instance Type	Defines vCPU, RAM, network, storage capabilities	t3.micro (2 vCPU, 1 GB RAM) vs m5.4xlarge (16 vCPU, 64 GB)
EBS Volume	Elastic Block Store — persistent disk attached to instance	100 GB gp3 SSD root volume
Security Group	Virtual firewall controlling inbound/outbound traffic	Allow port 22 (SSH) from my IP, port 443 from anywhere
Key Pair	RSA/ED25519 key for SSH access to Linux instances	my-server-key.pem
Elastic IP	Static public IPv4 address you own	52.12.34.56 — stays even if instance stops/starts

Term	Definition	Example
VPC	Virtual Private Cloud — isolated network for your instances	10.0.0.0/16 VPC with public and private subnets
User Data	Script run once on first boot to configure the instance	Install Apache, clone repo, start service
Instance Store	Ephemeral local storage — lost when instance stops	NVMe SSD for temp data, high-speed scratch space
Placement Group	Strategy for how instances are placed on hardware	Cluster (low latency), Spread (high availability), Partition (Hadoop)
ENI	Elastic Network Interface — virtual NIC attached to instance	Secondary IP for HA failover
Tenancy	How the hardware is shared	Shared (default), Dedicated Instance, Dedicated Host

Instance Families

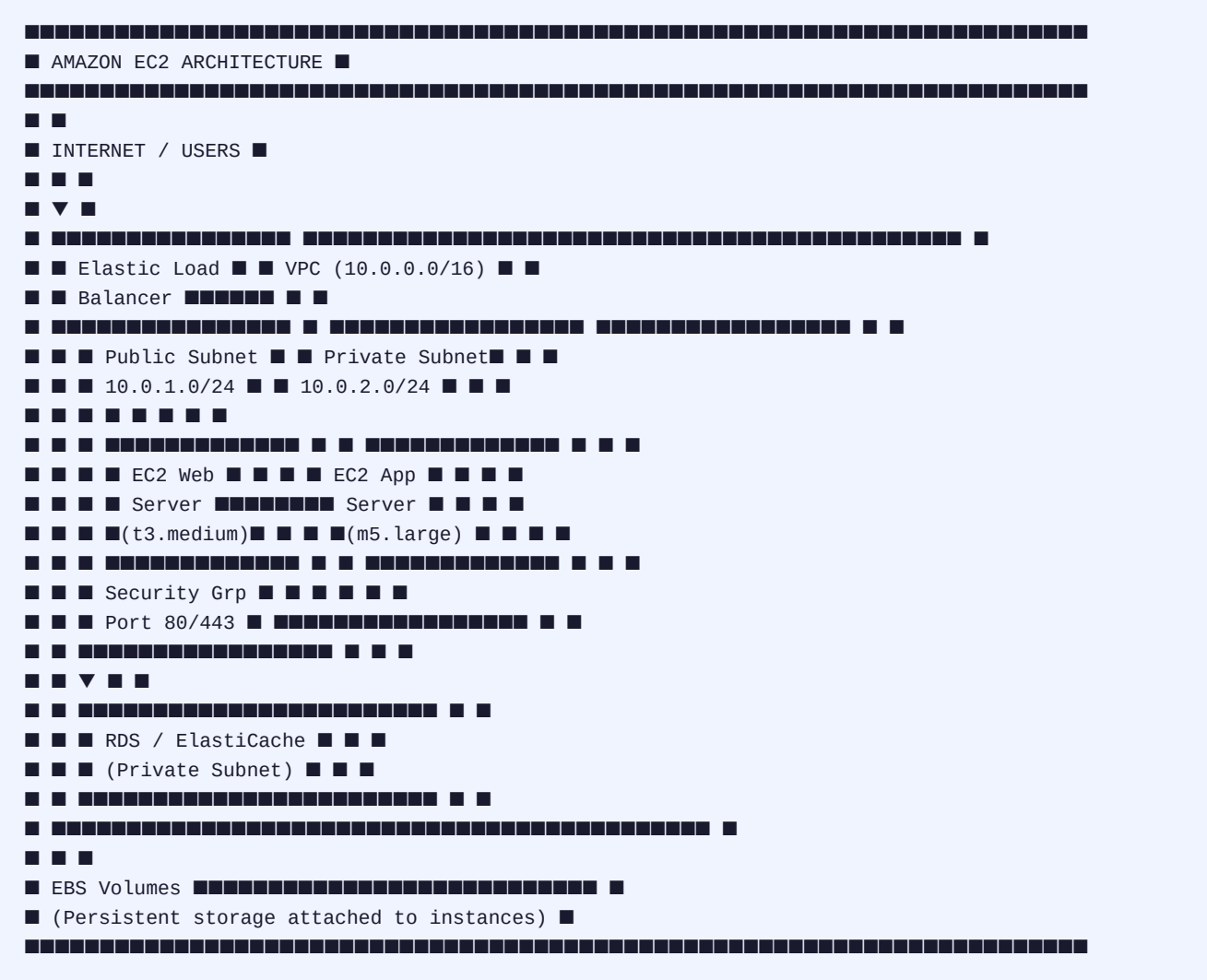
Family	Optimised For	Instance Types	Use Case
General Purpose	Balanced CPU/RAM	t3, t4g, m5, m6i, m7g	Web servers, dev/test, small DBs
Compute Optimised	High CPU performance	c5, c6i, c7g	Web crawling, video encoding, HPC
Memory Optimised	Large in-memory datasets	r5, r6i, x2gd, u-6tb1	In-memory DBs, Redis, SAP HANA
Storage Optimised	High disk throughput/IOPS	i3, i4i, d3, h1	NoSQL DBs, data warehouses, HDFS
Accelerated (GPU)	GPU compute	p3, p4d, g4dn, g5, inf2	ML training, inference, rendering
HPC Optimised	High-bandwidth networking	hpc6a, hpc7g	Tightly coupled HPC simulations
Arm / Graviton	Best price/performance	m7g, c7g, r7g, t4g	Any workload — up to 40% cheaper

Purchasing Options

Option	Description	Discount	Best For
On-Demand	Pay per second, no commitment	0%	Unpredictable workloads, testing
Reserved (1-yr)	1-year term commitment	~30-40%	Steady-state production workloads
Reserved (3-yr)	3-year term commitment	~50-60%	Long-running stable applications

Option	Description	Discount	Best For
Savings Plans	Flexible hourly commitment (any type)	~50-60%	Flexible RI alternative
Spot Instances	Bid on spare AWS capacity	~60-90%	Fault-tolerant, flexible timing
Dedicated Host	Physical server you control fully	0-30%	BYOL, compliance requirements
Dedicated Inst.	Instance on single-tenant hardware	~5-10%	Compliance without host control

3. Architecture & Working



Instance Launch Workflow

Step	Action	Detail
1	Choose AMI	Select OS image: Amazon Linux, Ubuntu, Windows, RHEL, or custom
2	Choose Type	Pick instance family and size based on CPU/RAM requirements

Step	Action	Detail
3	Configure Network	Select VPC, subnet, assign public IP, IAM role, placement
4	Add Storage	Attach EBS volumes (root + data); choose gp3, io2, st1, sc1
5	Configure SGs	Set inbound/outbound firewall rules for the instance
6	Add User Data	Optional bootstrap script to run on first launch
7	Launch	EC2 starts; hypervisor assigns physical hardware; AMI copied
8	Connect	SSH (Linux) or RDP (Windows); or EC2 Instance Connect

4. Features

Feature	Description	Use Case
Auto Scaling	Automatically add/remove instances based on CPU, memory, schedule, or custom metrics	Handle peak traffic on Black Friday without manual intervention
Elastic Load Balancing	Distribute traffic across multiple EC2 instances for HA and scale	Web tier with ALB sending traffic to 10 EC2 instances
EC2 Image Builder	Automated pipeline to build, test, and distribute custom AMIs	Golden AMI with patched OS and pre-installed software
EC2 Fleet	Launch diverse mix of instance types and purchase options in one API call	Get cheapest Spot capacity across m5, m6i, m5a instance types
Hibernate	Save in-memory RAM state to EBS; resume from exactly where it left off	Long-running computation that spans multiple sessions
Nitro System	Custom AWS silicon hypervisor delivering bare-metal-like performance	Near-zero virtualisation overhead for compute-intensive workloads
Nitro Enclaves	Isolated compute environments for processing sensitive data	HIPAA data, cryptographic operations, secrets processing
Serial Console	Emergency access to instance console without network access	Debug boot failures, network misconfigurations
Instance Metadata Service v2	Secure API to retrieve instance metadata from within (IMDSv2)	Get instance region, IP, IAM role credentials from within app
EBS Snapshots	Point-in-time backups of EBS volumes stored in S3	Disaster recovery, AMI creation, volume duplication
CloudWatch Agent	Push OS-level metrics (memory, disk) and logs to CloudWatch	Monitor RAM usage — not available natively in EC2 metrics

5. Real-World Use Cases

Industry	Use Case	Instance Choice
Startup	Host a full-stack web app	t3.small (dev), m5.large (prod)
E-Commerce	Auto-scaling web tier for seasonal peaks	c5.xlarge + Auto Scaling Group
Banking	Core banking application with compliance isolation	Dedicated Host + m5.4xlarge
Healthcare	HIPAA-compliant patient data processing	r5.2xlarge + encrypted EBS
AI / ML	Deep learning model training	p3.8xlarge (4x V100 GPUs)
Media	Video transcoding farm	c5.4xlarge Spot Instances
Gaming	Game server fleet for online multiplayer	c5n.2xlarge (low latency networking)
Enterprise	SAP HANA in-memory database	u-6tb1.56xlarge (6 TB RAM)
Research	HPC simulation cluster	hpc6a.48xlarge + Cluster Placement Group
DevOps	Jenkins CI/CD build agents	m5.2xlarge Spot + Auto Scaling

6. Practical Example

Scenario: Launch a secure web server on EC2 with proper networking, a bootstrap script, and a security group using the AWS CLI.

Step 1 – Create Security Group:

```
aws ec2 create-security-group \ --group-name web-sg \ --description 'Web server security group' \
--vpc-id vpc-0abc12345 # Allow HTTP, HTTPS, SSH aws ec2 authorize-security-group-ingress \
--group-id sg-xyz789 \ --protocol tcp --port 80 --cidr 0.0.0.0/0 aws ec2
authorize-security-group-ingress \ --group-id sg-xyz789 \ --protocol tcp --port 443 --cidr
0.0.0.0/0 aws ec2 authorize-security-group-ingress \ --group-id sg-xyz789 \ --protocol tcp --port
22 --cidr YOUR_IP/32
```

Step 2 – Launch EC2 Instance with User Data:

```
# user-data.sh — runs on first boot #!/bin/bash yum update -y yum install -y httpd systemctl start
httpd systemctl enable httpd echo "Hello from $(hostname)" > /var/www/html/index.html # Launch the
instance aws ec2 run-instances \ --image-id ami-0abcdef1234567890 \ --instance-type t3.medium \
--key-name my-key-pair \ --security-group-ids sg-xyz789 \ --subnet-id subnet-pub001 \
--associate-public-ip-address \ --iam-instance-profile Name=EC2-S3-Role \ --user-data
file://user-data.sh \ --block-device-mappings '[{"DeviceName":"/dev/xvda",
"Ebs":{"VolumeSize":30,"VolumeType":"gp3","Encrypted":true}}]' \ --tag-specifications
'ResourceType=instance,Tags=[{Key=Name,Value=web-server-01}]'
```

■ **Result:** EC2 instance is running Apache on port 80. The security group allows only your IP for SSH and the world for HTTP/HTTPS. EBS root volume is encrypted. IAM instance profile allows the app to access S3 without hardcoded credentials.

7. Integrations

AWS Service	How It Integrates
Amazon EBS	Persistent block storage attached to EC2 — root OS volume and data volumes
Elastic Load Balancing	ALB/NLB/GWLB distributes traffic across multiple EC2 instances
Auto Scaling	Automatically scales EC2 fleet based on CloudWatch metrics or schedules
Amazon VPC	EC2 lives inside VPC — subnets, route tables, NACLs, security groups control networking
AWS IAM	Instance profiles grant EC2 access to S3, DynamoDB, SSM without credentials
Amazon CloudWatch	Monitor CPU, network, disk metrics; set alarms; collect logs
AWS Systems Manager	Patch management, remote commands, parameter store — manage fleets at scale
Amazon RDS	EC2 app tier connects to RDS DB tier — classic 3-tier architecture
Amazon S3	EC2 reads and writes data to S3; EBS snapshots stored in S3
AWS Certificate Manager	ACM TLS certs used by ALB in front of EC2 instances
AWS Backup	Centralised backup of EBS volumes across EC2 fleet
Amazon Route 53	DNS routing to EC2 Elastic IPs or ELB endpoints

8. Comparison

Feature	EC2	Lambda	ECS Fargate	Elastic Beanstalk	Lightsail
Control	Full OS	Code only	Container	Platform	Limited
Management	You manage OS	Zero	Low	Managed PaaS	Very simple
Scaling	Manual/ASG	Automatic	Auto	Auto	Manual
Max Duration	Unlimited	15 min	Unlimited	Unlimited	Unlimited
Pricing	Per second	Per invocation	Per vCPU/mem	Per EC2	Flat monthly
Custom runtime	■ Any	Limited	■ Container	■ Via EC2	■ No
Use when	Full control needed	Event-driven tasks	Containerised apps	Quick deploy PaaS	Simple websites

λ AWS Lambda

Run Code Without Servers — Event-Driven Serverless Functions

1. Simple Introduction

AWS Lambda is a **serverless compute service** that lets you run code without provisioning or managing any servers. You upload your code (a **function**), define what triggers it, and AWS handles everything else — scaling, patching, high availability. You pay only for the compute time your code actually uses, measured in milliseconds.

Why Does It Exist? Most applications spend most of their time waiting — for a request to arrive, for a database to respond, for a user to act. Traditional servers sit idle during this time but still cost money. Lambda solves this: compute resources only exist for the milliseconds your code runs.

■ **Real-World Analogy:** Lambda is like a **vending machine**. When you press a button (trigger), the machine springs to life, dispenses your item (runs your code), and goes dormant again. You don't own the machine's motor — you just pay per dispense. 1,000 people pressing buttons simultaneously? 1,000 vending actions happen in parallel, automatically. No manager needed.

Business Problems Solved:

- Zero server management — no OS patching, capacity planning, or infrastructure
- Automatic scaling — from 0 to 10,000 concurrent executions in seconds
- Pay-per-use pricing — no charge when code is not running
- High availability built-in — Lambda runs across multiple AZs automatically
- Supports 12+ runtimes — Python, Node.js, Java, Go, Ruby, .NET, custom runtimes

2. Core Concepts

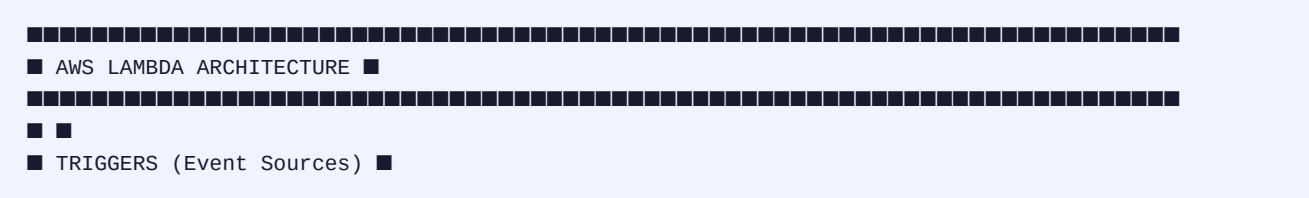
Term	Definition	Example
Function	The unit of Lambda — your code + configuration	process_order.py uploaded as a Lambda function
Handler	The entry-point function Lambda calls	"def handler(event, context):" in Python
Event	JSON payload passed to your function when triggered	{"Records": [{"s3": {"bucket": {...}}}]}
Context	Object with metadata about the invocation	context.aws_request_id, context.get_remaining_time_in_millis()
Trigger	What causes the function to execute	API Gateway request, S3 upload, SQS message, schedule
Invocation	One execution of a Lambda function	Function called 1 million times = 1 million invocations
Concurrent Exec.	Number of function instances running simultaneously	1000 API requests at once = up to 1000 concurrent executions

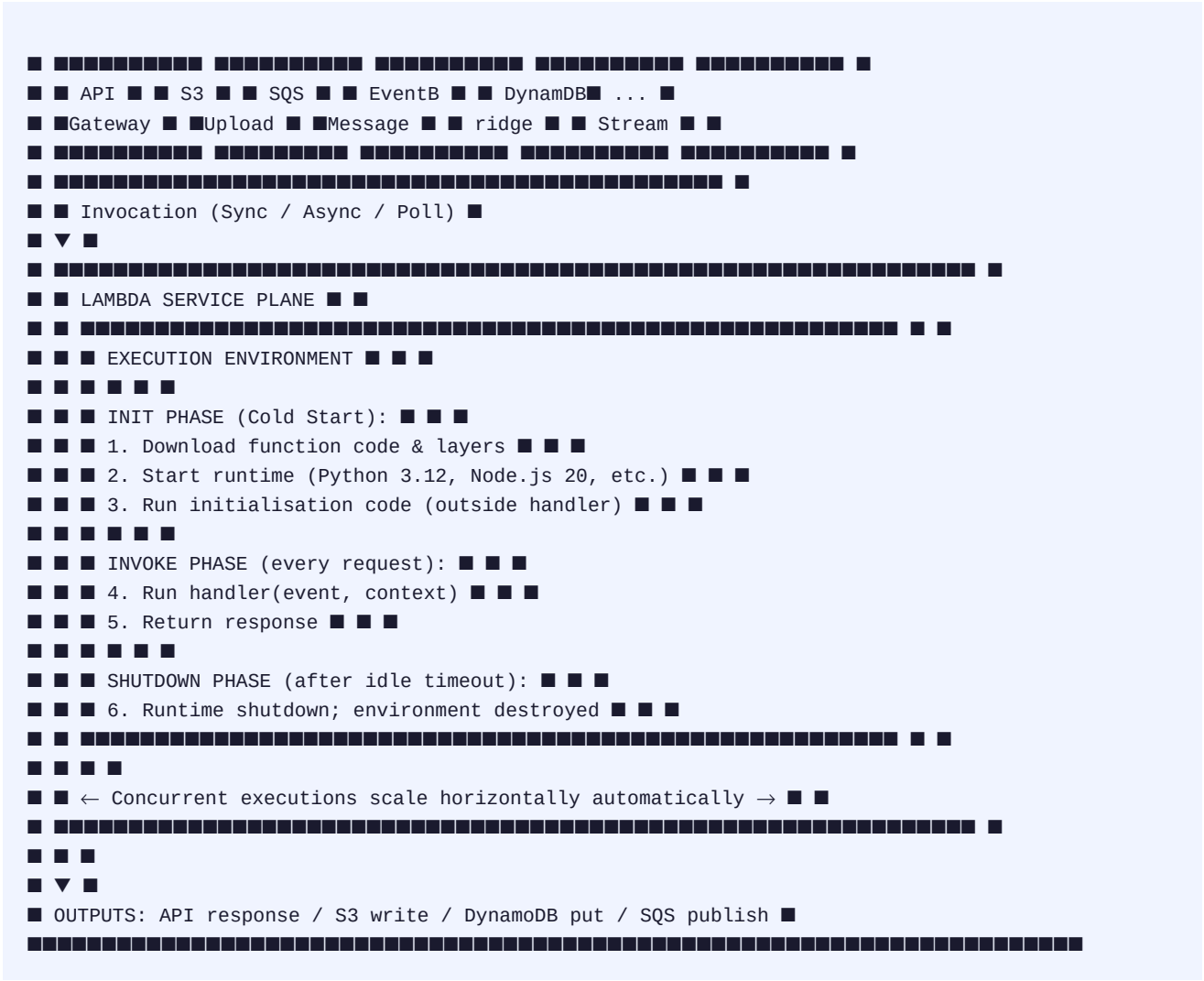
Term	Definition	Example
Cold Start	Latency when Lambda initialises a new execution environment	First request after idle: ~100ms–5s extra depending on runtime
Warm Start	Reuse of existing execution environment — fast	Subsequent requests: < 1ms overhead
Execution Env.	Isolated container-like sandbox Lambda runs in	Reused for warm invocations; destroyed after idle period
Layers	Reusable code/libraries packaged separately from function code	numpy, pandas layer shared across 20 functions
Environment Vars	Key-value config passed into function at runtime	DB_HOST=mydb.cluster-abc.us-east-1.rds.amazonaws.com
Memory	Allocated RAM (128 MB – 10,240 MB); CPU scales proportionally	1024 MB = ~0.5 vCPU; 1769 MB = 1 full vCPU
Timeout	Max time a function can run (1 sec – 15 min)	Set to 30 seconds for API handler, 900 seconds for batch job
Reserved Concurrency	Hard cap on concurrent executions for this function	Set to 100 to prevent DB overload
Provisioned Concurrency	Pre-warm N execution environments to eliminate cold starts	50 warm instances for a latency-sensitive API
Destinations	Route async invocation results to SQS, SNS, Lambda, or EventBridge	On success → SQS queue; on failure → SQS DLQ
Function URL	Dedicated HTTPS endpoint for a function (no API Gateway needed)	https://abc123.lambda-url.us-east-1.on.aws/

Invocation Types

Type	How It Works	Response	Example Trigger
Synchronous	Caller waits for result	Returns output immediately	API Gateway, ALB, SDK direct call
Asynchronous	Lambda queues event; caller doesn't wait	202 Accepted	S3 events, SNS, EventBridge
Event Source Mapping	Lambda polls the source and processes batches	Batch result	SQS, Kinesis, DynamoDB Streams, Kafka

3. Architecture & Working





Cold Start vs Warm Start

Phase	Cold Start	Warm Start
When	First invocation / new concurrent instance needed	Subsequent requests to existing environment
Steps	Download code → Init runtime → Run init code → Handler	Only Handler runs
Latency	100ms – 5+ seconds (varies by language/size)	< 1ms overhead
Mitigation	Provisioned Concurrency, smaller packages, compiled langs	Already fast
Cost	Normal (init time billable from Dec 2023)	Normal

4. Features

Feature	Description	Use Case
Automatic Scaling	Scales from 0 to 10,000+ concurrent executions instantly	Handle 0 or 100,000 requests — same code, no config change
Lambda Layers	Reusable ZIP archives of libraries/code shared across functions	pandas layer used by 50 data-processing Lambda functions
Container Image Support	Deploy functions as Docker images up to 10 GB	Large ML models, complex dependencies via custom container
Lambda@Edge / CloudFront	Run Lambda at CloudFront edge locations globally	A/B testing, URL rewriting, auth at the CDN edge
VPC Support	Lambda accesses private resources inside your VPC	Function accesses RDS database in private subnet
SnapStart (Java)	Pre-initialise Java functions; resume from snapshot — near-zero cold start	Java Spring Boot Lambda with < 1 second cold start
Provisioned Concurrency	Pre-warm N environments to eliminate cold starts	Latency-sensitive API requiring sub-100ms responses
Function URLs	Dedicated HTTPS endpoint without API Gateway	Simple webhook receiver, internal tools
Code Signing	Ensure only trusted, signed code runs in Lambda	Compliance requirement — only verified deployments
AWS Graviton2/3	arm64 architecture — 19% better price/performance	Cost-optimised Lambda functions with same performance
Ephemeral Storage (/tmp)	512 MB to 10,240 MB of temporary disk storage per invocation	Downloading and processing large files within a function
Recursive Loop Detection	Automatically detects and stops Lambda-to-Lambda-to-Lambda loops	Prevents runaway costs from infinite recursion bugs

5. Real-World Use Cases

Industry	Use Case	Trigger
E-Commerce	Process new orders, update inventory, send confirmation email	API Gateway POST /orders
Banking	Real-time fraud scoring on every transaction	Kinesis stream (every txn)
Healthcare	FHIR API for patient data — serverless REST API	API Gateway
Media	Auto-generate thumbnails on image upload	S3 PutObject event
IoT	Process sensor readings, trigger alerts for anomalies	IoT Rule → Lambda

Industry	Use Case	Trigger
Startup	Entire backend as serverless functions	API Gateway
Enterprise	Nightly report generation and email distribution	EventBridge (cron)
AI / ML	ML inference API — run predictions on demand	API Gateway + Lambda + SageMaker
DevOps	Auto-remediation — restart failed ECS tasks on CloudWatch alarm	CloudWatch Alarm
Gaming	Leaderboard API — read/write player scores	API Gateway

6. Practical Example

Scenario: Build a serverless image thumbnail generator. When an image is uploaded to an S3 bucket, Lambda automatically creates a 150x150 thumbnail and saves it to a separate bucket.

Lambda Function Code (Python):

```
import boto3 from PIL import Image import io, os s3 = boto3.client('s3') THUMB_BUCKET = os.environ['THUMB_BUCKET'] def handler(event, context): # Extract S3 event details record = event['Records'][0]['s3'] src_bucket = record['bucket']['name'] key = record['object']['key'] # Download original image from S3 response = s3.get_object(Bucket=src_bucket, Key=key) img_data = response['Body'].read() # Create thumbnail using Pillow img = Image.open(io.BytesIO(img_data)) img.thumbnail((150, 150)) # Upload thumbnail to destination bucket buf = io.BytesIO() img.save(buf, format='JPEG', quality=85) buf.seek(0) s3.put_object( Bucket=THUMB_BUCKET, Key=f'thumbs/{key}', Body=buf, ContentType='image/jpeg' ) return {'statusCode': 200, 'body': f'Thumbnail created: thumbs/{key}'}
```

Deploy with AWS CLI:

```
# Package (Pillow layer already attached) zip -r function.zip lambda_function.py # Create the function aws lambda create-function \ --function-name image-thumbnailer \ --runtime python3.12 \ --role arn:aws:iam::123456789:role/LambdaS3Role \ --handler lambda_function.handler \ --zip-file fileb://function.zip \ --memory-size 512 \ --timeout 30 \ --environment 'Variables={THUMB_BUCKET=my-thumbnails}' # Add S3 trigger aws lambda add-permission \ --function-name image-thumbnailer \ --statement-id s3-invoke \ --action lambda:InvokeFunction \ --principal s3.amazonaws.com \ --source-arn arn:aws:s3:::my-uploads aws s3api put-bucket-notification-configuration \ --bucket my-uploads \ --notification-configuration '{ "LambdaFunctionConfigurations": [{ "LambdaFunctionArn": "arn:aws:lambda:us-east-1:123:function:image-thumbnailer", "Events": ["s3:ObjectCreated:*"] }] }'
```

■ **Result:** Every image uploaded to 'my-uploads' S3 bucket automatically triggers Lambda. Thumbnails appear in 'my-thumbnails' bucket within seconds. Cost for 1 million thumbnails with 512MB Lambda = ~$0.50 total (vs always-on EC2 = ~$30/month minimum).

7. Integrations

AWS Service	Integration Pattern
Amazon API Gateway	Lambda as the backend for REST/HTTP/WebSocket APIs — the #1 Lambda pattern
Amazon S3	S3 events trigger Lambda on create/delete/restore; Lambda reads/writes S3

AWS Service	Integration Pattern
Amazon DynamoDB	Lambda triggered by DynamoDB Streams; Lambda reads/writes DynamoDB tables
Amazon SQS	Lambda polls SQS queue, processes messages in batches, scales with queue depth
Amazon SNS	SNS pushes messages to Lambda; Lambda sends SNS notifications
Amazon EventBridge	EventBridge rules invoke Lambda on schedule or event match
Amazon Kinesis	Lambda processes streaming records from Kinesis Data Streams in real time
AWS Step Functions	Lambda runs as a Task state in Step Functions workflows
Amazon Cognito	Lambda triggers on sign-up, sign-in, pre-token — custom auth flows
Amazon CloudWatch	Lambda logs go to CloudWatch Logs; Lambda invoked by CloudWatch alarms
Amazon RDS Proxy	Lambda uses RDS Proxy to pool DB connections (Lambda can't hold connections)
AWS AppConfig	Lambda reads feature flags from AppConfig for safe feature rollouts

8. Comparison

Feature	Lambda	EC2	ECS Fargate	App Runner	Cloud Run (GCP)
Management	Zero (serverless)	You manage OS	Container only	Zero	Zero
Scaling	Auto (0→10k)	Manual/ASG	Auto	Auto	Auto
Max Duration	15 minutes	Unlimited	Unlimited	Unlimited	60 minutes
Cold Start	Yes (mitigable)	None (always on)	Faster than Lambda	Yes	Yes
Pricing	Per ms + requests	Per second	Per vCPU/mem/s	Per vCPU/mem/s	Per ms
State	Stateless	Stateful	Stateless	Stateless	Stateless
GPU Support	■ No	■ Yes	■ No	■ No	■ Yes
Best For	Event-driven, short tasks	Full control workloads	Containerised services	Container APIs	Serverless containers

■ AWS Serverless Application Repository

Discover, Deploy, and Share Serverless Applications Instantly

1. Simple Introduction

The AWS Serverless Application Repository (SAR) is a **managed repository for serverless applications**. It lets you discover, deploy, and publish serverless applications built with AWS services like Lambda, API Gateway, DynamoDB, and SQS — without writing code from scratch. Think of it as an **app store for serverless components**.

Why Does It Exist? Developers repeatedly build the same serverless patterns — image resizers, authentication systems, Slack bots, S3 lifecycle managers, etc. SAR prevents this wheel-reinvention by allowing teams to publish, share, and deploy battle-tested serverless applications with a single click or API call.

■ **Real-World Analogy:** SAR is like the **App Store / Google Play for serverless infrastructure**. Developers (publishers) submit their serverless apps. Other developers (consumers) browse, review, and install them into their own AWS accounts. A thumbnail generator that took one team 2 days to build can be deployed by another team in 60 seconds — with full customisation through parameters.

Business Problems Solved:

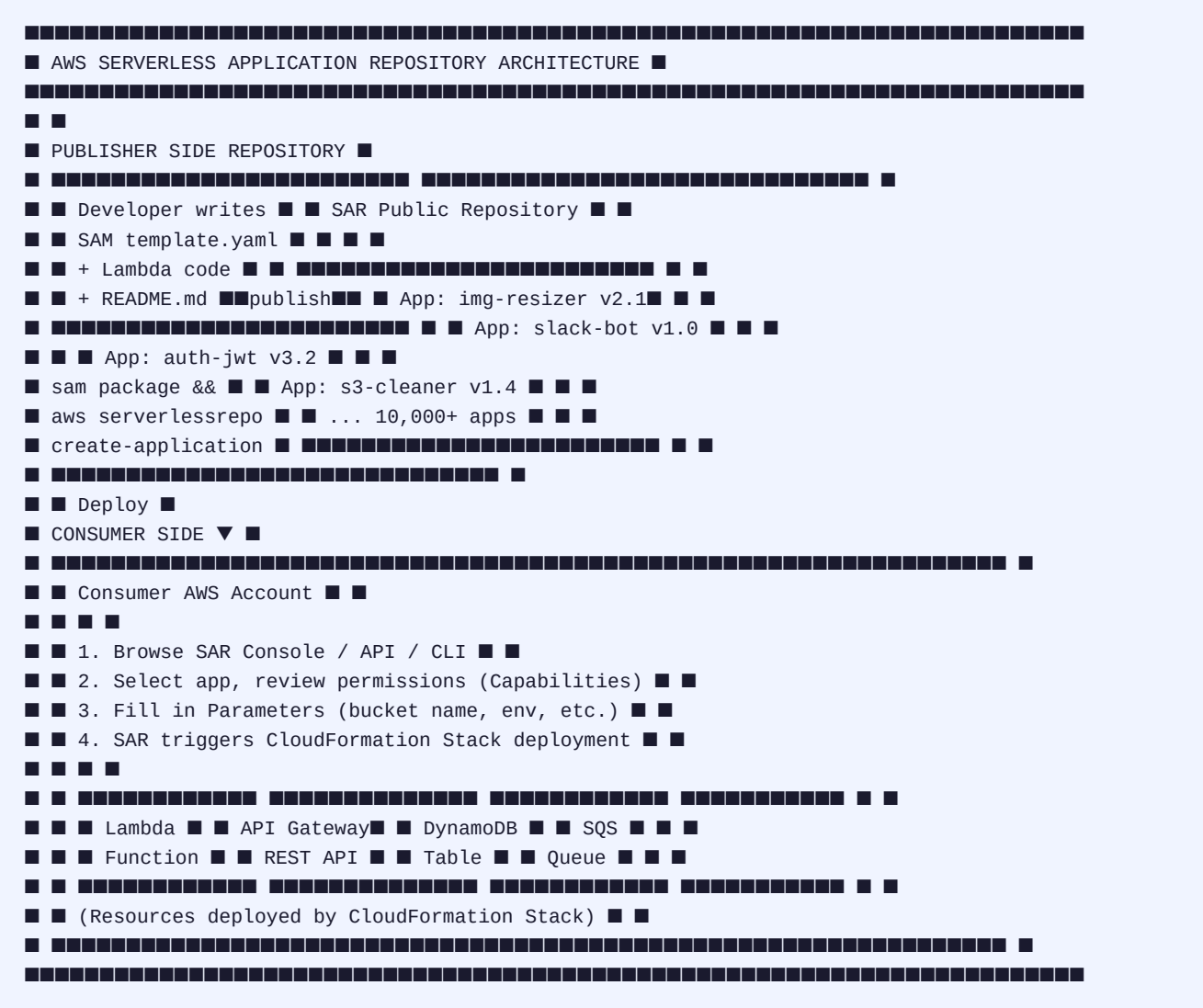
- Eliminate duplicate effort — don't rebuild common serverless patterns
- Accelerate time-to-market — deploy proven apps in minutes, not days
- Standardise internal components — publish company-approved patterns for teams to reuse
- Monetise serverless solutions — third-party ISVs publish paid/free apps
- Safe deployment — SAR uses CloudFormation/SAM under the hood for consistent deployments

2. Core Concepts

Term	Definition	Example
Application	A packaged serverless solution ready to deploy	S3 Thumbnail Generator, Slack Notifier, Auth0 Authorizer
SAM Template	AWS Serverless Application Model — CloudFormation extension defining serverless apps	template.yaml with Lambda + API GW + DynamoDB defined
Publisher	AWS account that submits an application to SAR	Your company or an ISV like Datadog, Splunk
Consumer	AWS account that deploys an application from SAR	Any developer in any AWS account
Sharing Policy	Controls who can see and deploy the application	Public (everyone), Private (your account), Shared (specific accounts)
Parameters	CloudFormation parameters to customise the app at deploy time	BucketName, Environment (dev/prod), MemorySize

Term	Definition	Example
Capabilities	Explicit acknowledgement of IAM permissions the app requires	CAPABILITY_IAM, CAPABILITY_NAMED_IAM, CAPABILITY_AUTO_EXPAND
Semantic Version	Version string for each app release (semver format)	1.0.0, 2.3.1, 3.0.0-beta
License	Open-source license declared for the app	MIT, Apache-2.0, GPL-3.0
Nested Apps	SAR applications can include other SAR apps as components	Deploy auth app + thumbnail app together as one stack
SAM	AWS Serverless Application Model — IaC framework that extends CloudFormation	sam build && sam deploy for packaging and deploying

3. Architecture & Working



Publishing Workflow

Step	Action	Detail
1	Build the app	Write Lambda functions + SAM template.yaml
2	Test locally	sam local invoke / sam local start-api
3	Package	sam package — uploads code to S3, generates packaged template
4	Publish to SAR	aws serverlessrepo create-application or sam publish
5	Set metadata	Name, description, author, labels, license, readme, sharing policy
6	Version it	Publish new version with semantic version string
7	Set sharing	Public (all AWS), Private (your account), or share with specific account IDs

Deployment Workflow (Consumer)

Step	Action	Detail
1	Discover	Browse SAR console, search by keyword/category, or use CLI
2	Review	Read description, review IAM permissions required, check license
3	Configure	Fill in CloudFormation parameters (customize the app)
4	Acknowledge	Accept required Capabilities (IAM resource creation acknowledgement)
5	Deploy	SAR triggers a CloudFormation stack in your account
6	Use	Resources created — invoke Lambda, call API, use the app

4. Features

Feature	Description	Benefit
Public & Private Apps	Publish apps as public (for everyone) or private (your account/org)	Share internally or contribute to the community
Semantic Versioning	Multiple versions of the same app; consumers choose the version	Safe updates — consumers stay on v1.x while you release v2.0
Nested Applications	SAR apps can reference other SAR apps as sub-components	Compose complex architectures from reusable building blocks
CloudFormation Native	SAR apps deploy via CloudFormation — full stack management	Update, rollback, delete entire app as one atomic operation
AWS Console Integration	Deploy from SAR directly within Lambda console 'Create function'	Discover apps without leaving the Lambda workflow
IAM Permission Preview	Consumers see all IAM permissions the app requires before deploying	Security review before accepting unknown app's permissions

Feature	Description	Benefit
License Declaration	Open source license declared on each app	IP compliance — know what license you're accepting
SAM Integration	First-class integration with AWS SAM framework for build/test/publish	One-command publish: sam publish
Parameter Overrides	Customize apps with CloudFormation parameters at deploy time	Same app, different configs for dev/staging/production
Application Insights	View download counts and deployment metrics for your published apps	Understand adoption of your serverless components

5. Real-World Use Cases

Who	Use Case	App Example
Enterprise Platform Team	Publish approved internal Lambda patterns for app teams to reuse	Auth middleware, logging helper, secrets rotation
ISV / SaaS Company	Distribute integration apps to customers	Datadog log forwarder, Auth0 JWT authorizer, Stripe webhook handler
Startup	Deploy pre-built serverless backend components to ship faster	CRUD API with DynamoDB, user auth with Cognito — deployed in minutes
E-Commerce	Add serverless capabilities without in-house expertise	Abandoned cart email trigger, product image resizer
Healthcare	HIPAA-compliant FHIR API pattern shared across hospital systems	HL7 FHIR R4 API deployed into each facility's own account
DevOps Teams	Standardise operational Lambda functions across 50 AWS accounts	CloudTrail → S3 archiver, EC2 unused instance notifier
Open Source Community	Share serverless utilities with the AWS community	S3 bucket cleaner, cost anomaly notifier, RDS snapshot manager
AI / ML Teams	Deploy ML inference endpoints quickly from shared pattern	SageMaker async inference trigger + result notifier

6. Practical Example

Scenario A — Consuming: Deploy a Slack notification Lambda from SAR that alerts a Slack channel whenever an S3 object is deleted.

Deploy via AWS CLI:

```
# Search for a Slack notifier app aws serverlessrepo search-applications \ --search-expression
'slack notification s3' # Get the application details (use ApplicationId from search results) aws
serverlessrepo get-application \ --application-id
arn:aws:serverlessrepo:us-east-1:177890581765:applications/Slack-Notifier # Create CloudFormation
change set to deploy it aws serverlessrepo create-cloud-formation-change-set \ --application-id
arn:aws:serverlessrepo:us-east-1:177890581765:applications/Slack-Notifier \ --semantic-version
1.0.0 \ --stack-name slack-s3-notifier \ --parameter-overrides
'[{"Name":"SlackWebhookUrl","Value":"https://hooks.slack.com/..."},
```

```
{"Name":"SlackChannel","Value":"#s3-alerts"}]' \ --capabilities CAPABILITY_IAM # Execute the change set (deploys the app) aws cloudformation execute-change-set \ --change-set-name \ --stack-name slack-s3-notifier
```

Scenario B — Publishing: Publish your own Lambda function to SAR.

```
# template.yaml (SAM template) Transform: AWS::Serverless-2016-10-31 Metadata: AWS::ServerlessRepo::Application: Name: s3-object-notifier Description: Sends SNS notification when S3 objects are created Author: Your Name SpdxLicenseId: MIT LicenseUrl: LICENSE.txt ReadmeUrl: README.md Labels: [s3, sns, notifications] SemanticVersion: 1.0.0 Parameters: SourceBucket: Type: String SNSTopicArn: Type: String Resources: NotifierFunction: Type: AWS::Serverless::Function Properties: Handler: app.handler Runtime: python3.12 Environment: Variables: SNS_TOPIC_ARN: !Ref SNSTopicArn Events: S3Upload: Type: S3 Properties: Bucket: !Ref SourceBucket Events: s3:ObjectCreated:* # Package and publish sam package --s3-bucket my-sar-package-bucket --output-template packaged.yaml sam publish --template packaged.yaml --region us-east-1
```

■ **Result:** Your app is now visible in the SAR console. Set it to Public and any AWS developer worldwide can find it by searching 's3 notifier' and deploy it into their account with their own parameters — no code sharing needed.

7. Integrations

AWS Service	How It Integrates
AWS Lambda	Every SAR app contains at least one Lambda function — Lambda is the core compute
AWS CloudFormation	SAR uses CloudFormation under the hood for all deployments — full IaC lifecycle
AWS SAM	SAM is the primary framework for building, testing, and publishing SAR apps
Amazon S3	Published app code is stored in SAR-managed S3 buckets; apps often use S3 as triggers
Amazon API Gateway	Many SAR apps expose REST/HTTP APIs via API Gateway
Amazon DynamoDB	Serverless apps in SAR commonly use DynamoDB as the database
AWS IAM	SAR apps declare required IAM permissions; consumers review before deploying
Amazon EventBridge	SAR apps triggered by EventBridge rules; some apps publish to EventBridge
AWS CodePipeline	SAR publishing as part of a CI/CD pipeline for internal platform components
Amazon CloudWatch	Deployed SAR apps automatically log to CloudWatch Logs via Lambda integration

8. Comparison

Feature	SAR	CloudFormation Modules	AWS Solutions Library	Terraform Registry	npm / PyPI
Type	Serverless app store	CF template fragments	AWS reference architectures	IaC module registry	Code package registry

Feature	SAR	CloudFormation Modules	AWS Solutions Library	Terraform Registry	npm / PyPI
Deploy method	1-click / CLI	Nested stack	Manual / CF / CDK	terraform apply	pip / npm install
Serverless focus	■ Yes (Lambda centric)	■ Any CF resource	■ Any architecture	■ Any infra	■ Code libraries
Public sharing	■ Yes	■ Limited	■ Yes (read only)	■ Yes	■ Yes
Versioning	■ Semantic versioning	Limited	■ No versions	■ Yes	■ Yes
AWS Native	■ First-party AWS	■ First-party AWS	■ First-party AWS	■ Third-party	■ Third-party
Best For	Deploy serverless apps	Reusable CF snippets	Reference architectures	IaC module sharing	Code library sharing

■ **When to Use SAR:** Use SAR when you want to quickly deploy a complete serverless application pattern into your AWS account. Use CloudFormation Modules for reusable infrastructure fragments. Use the AWS Solutions Library for reference architectures. Use Terraform Registry for multi-cloud IaC reuse.

Quick-Reference Summary

Service	Type	Key Strength	Max Duration	Best For
■ AWS Batch	Managed batch scheduler	Job queues + Spot + GPU + array jobs	Unlimited	Long compute-heavy batch work
■ Amazon EC2	Virtual servers	Full OS control + 750+ instance types	Unlimited	Any workload needing persistent
λ AWS Lambda	Serverless functions	Zero server mgmt + auto-scale to 10k+	15 minutes	Short event-driven tasks
■ SAR	Serverless app store	1-click deploy of proven patterns	N/A	Quickly adopt community/interna apps

Decision Framework — Which Compute to Choose?

```
START: What is your workload?
■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■ ■
▼ ▼ ▼
Batch / Offline Event-Driven Always-On Service
Processing Short Tasks / Full Control
■ ■ ■
▼ ▼ ▼
■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■
■Need GPU or ■ ■Duration ■ ■Need OS access / ■
■>15 min jobs? ■ ■< 15 minutes? ■ ■persistent state? ■
■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■
■YES ■YES ■YES
▼ ▼ ▼
AWS BATCH AWS LAMBDA AMAZON EC2
(or EC2+ASG) (Serverless) (or ECS/EKS)

AND / OR
Deploy from SAR → Use pre-built serverless patterns
to accelerate time-to-deployment for any Lambda workload
```